

Kanon M1: Foundations and the Verification Core

RedStone Oracle Adaptor, the one implementation both modes will call

v1.1

2026-09-16

Equilibrium Group Oy
Meritullinkatu 1
Helsinki
Finland

Logos
RFP-020

TABLE OF CONTENTS

Scope	3
1. What M1 delivered, in one page	4
2. The verification core: the life of one payload	5
2.1 Decoding the RedStone format	5
2.2 Signatures and the M-of-N threshold	6
2.3 One round, one price	6
2.4 Time, staleness and replay	6
2.5 Asset identity and value sanity	7
2.6 Typed errors, shared by both modes	7
3. Publishing the verified price	8
4. What an update costs	10
5. How we know it works	12
5.1 Conformance against payloads RedStone signed	12
5.2 Property tests: claims about every input	12
5.3 The accept-and-reject contract, in one suite	12
5.4 The traceability matrix is machine-checked	12
5.5 The gates and the sequencer	13
6. What M1 deliberately does not contain	14
6.1 The authorised-signer roster is M2, by design	14
7. What we need from Logos	16
7.1 A SPEL release, which would settle the LEZ pin with it	16
7.2 Where the admin-authority extension is going to live	17
7.3 Two licensing questions, one cheap and one positional	17
7.4 The canonical price account is harder to depend on than it needs to be	18
7.5 The upgrade question underneath the price account	18
8. Reproducing this	19

Author: Equilibrium Group Oy

Status: Milestone deliverable

Created: 2026-09-12

Label: verification-core

Client: Logos

Target: RFP-020 milestone M1, the twenty-seven tasks M1-01 to M1-26 and M1-04a

Scope

M1 is the build milestone that follows M0's measurement: the repository, its CI gates, and above all the verification core — the one audited implementation of RedStone payload verification that both delivery modes, push and pull, will call. This report states what each piece was for, what was built, and how it is proven, without descending into the code. It also states, deliberately, what M1 does *not* contain, because the repository's layout makes that visible and it is easier to read the boundary here than to infer it there.

The organising idea throughout: nothing in this milestone is claimed without a test or a CI gate behind it. Every behaviour described in section 2 is asserted by the test suite; every figure in section 4 is generated by the code that asserts it; and the requirement-by-requirement mapping is itself machine-checked (section 5.4), so a claim of completeness cannot quietly outrun the evidence.

Full detail lives in the delivered repository: `TRACEABILITY.md` maps every RFP-020 requirement to its implementation and its evidence, `adr/` records the twenty-nine design decisions and why, and `COSTS.md` is the cost table in full.

1. What M1 delivered, in one page

The verification core is implemented, tested and priced. Given a signed RedStone payload, a feed configuration and the LEZ clock, `verifier-core` either returns a verified median price on the canonical Q64.64 scale or a typed error naming exactly what failed. It is `no_std`, free of `unsafe`, panic-free on any input, and depends on neither delivery mode — which is what lets one audited implementation back both.

All twenty-seven M1 tasks are complete. In outline:

area	what stands
Verification core	decode, signature recovery, M-of-N threshold, one-round median, staleness against the LEZ clock, asset identity, value sanity, Q64.64 conversion, typed errors — section 2
Price-account write	the RFP-019 canonical account populated field by field, with update rules that stop a price moving backwards — section 3
Cost accounting	<code>verify_feed</code> measured component by component in the product guest, published in <code>COSTS.md</code> , asserted in CI — section 4
Evidence	conformance against payloads RedStone signed, property tests, a contract-shaped accept-and-reject suite, and a machine-checked traceability matrix — section 5
Infrastructure	dual licence with a CI-enforced allowlist, lint/build/test/ <code>no_std</code> gates, and a standalone LEZ sequencer booted in CI — section 5.5

The numbers: **211 tests, all green**, across eight test targets, plus property tests that generate inputs rather than choose them. A three-signer update — RFP-020's default threshold — costs **1,831,638 cycles** inside the product guest, 5.5% of LEZ's 33.5M per-transaction budget, with `secp256k1` recovery at 95.85% of it; each additional signer costs about 605,000 cycles. These figures confirm M0's finding on the shipped code path rather than a harness.

Twenty-nine architecture decision records document the design. Five questions are open with Logos (section 7); none blocks M2.

What M1 is not: the milestone builds the engine, not the vehicles. The crates for pull-mode verification, the SDK, the relay and the reference consumers exist as deliberate placeholders that name the milestone that fills them — section 6 draws that boundary precisely, including where the authorised-signer roster will live (M2, not M1).

2. The verification core: the life of one payload

`verifier-core` is the crate RFP-020's F2 asks for — verification packaged as a library callable both from the push aggregator and from third-party consumer programs, so one audited implementation backs both modes. Its single entry point takes a payload, a feed configuration and a time source, and answers with a verified price or a typed error. That it is the **only** crate that verifies, and that it depends on neither mode, is the structural decision the rest of this section rests on (ADR 2): push and pull cannot disagree about a signer set or a staleness window, because there is nothing for them to disagree with. This section walks a payload through the checks in the order the code runs them; each subsection states the goal, what was built, and the decision record behind it.

Two properties hold for the whole crate rather than any one check, and both are enforced in CI rather than by review. It is **no_std**, **unsafe-free** and **panic-free** (ADR 4): inside a zkVM guest a panic aborts the transaction rather than rejecting the package, so every malformed input must produce an error, not a crash. And its cryptography sits behind a **backend trait** (ADR 3): `secp256k1` recovery and `keccak256` are pluggable, so if LEZ later ships a recovery precompile — the follow-on M0 recommended — adopting it is a localised change, not a rewrite. Worth stating the limit of that promise, since it is the kind of claim that grows in the retelling: the trait absorbs a change of **implementation** behind the same primitives. A change of **scheme** — a curve or hash other than `secp256k1` and `keccak256` — is not something it buys, because RedStone's wire format names those primitives and the decoder is written to it.

2.1 Decoding the RedStone format

Goal. Read RedStone's binary data-package format — asset identifier, value, timestamp, signature — without trusting a byte of it.

Built. A first-party decoder, written from RedStone's wire format and kept separate from verification (ADR 5). It bounds-checks every length and is tested against truncation at every single offset of a real payload: any cut, anywhere, is a clean rejection. The decoder also enforces a maximum payload size (ADR 26), and packages for feeds other than the requested one are skipped without being paid for, so **a payload cannot choose how much verification costs** (ADR 25) — a submitter cannot inflate a transaction's cycle bill by padding the payload.

Proven. Conformance against payloads RedStone actually published, using their own signatures as the oracle — section 5.1.

2.2 Signatures and the M-of-N threshold

Goal. A price exists only when at least M of the N authorised signers vouch for it (F3, default 3-of-N).

Built. Each package's signer is recovered from its secp256k1 signature and looked up in the configured signer set. The threshold counts **distinct authorised signers**: one slot per configured signer, so a signer counts once no matter how many packages it sends, and one package counts once no matter how many times it repeats the feed. A duplicate package is skipped rather than treated as fatal (ADR 24), so an attacker who can copy bytes — and any relayer can — cannot deny the feed by appending a copy of an honest package. A package from a signer *not* in the configured set, by contrast, rejects the payload outright: strangers are a fault, never a no-op.

Proven. The threshold is pinned at every boundary an off-by-one could live in: M, M-1, a threshold of one, and unanimity. Property tests generate payloads rather than choosing them — section 5.2.

2.3 One round, one price

Goal. The price handed to a consumer must come from a single oracle round, not a splice of old and new data.

Built. All counted packages must share one timestamp — the round the most distinct configured signers agree on — and the verified price is the **median** across those signers, computed by RedStone's own definition so that this adaptor and RedStone's EVM contracts agree on what a payload is worth (ADR 15, ADR 27). A payload mixing rounds for the requested feed is refused, which is what closes off replay-splicing: an attacker holding yesterday's signed packages cannot blend them with today's to move the median.

2.4 Time, staleness and replay

Goal. A package older than the feed's `maxAge` must never produce a price, and “older” must be measured against a clock the payload cannot influence (F4).

Built. Staleness is measured against LEZ's own on-chain clock program, pinned by address and updated every block (ADR 13), read through a `TimeSource` trait by the small `kanon-clock` crate — a crate rather than a module so that a pull consumer can supply its own clock without linking the push path (ADR 18). The validity window is **two-sided** (ADR 18): too old is `StalePackage`, but too far in the future — beyond a small skew allowance — is `FuturePackage`, a distinct failure, because a future timestamp is evidence of a misbehaving source rather than a slow relayer. The window's width is itself bounded from above (ADR 29): a misconfigured `maxAge` large enough to disable the check is refused at configuration time, so the staleness rule cannot be configured out of existence and look normal.

Every package carrying the requested feed must be inside the window — one stale package rejects the payload even when the remaining signers meet the threshold, which is ADR 15's strict-rejection rule: a quorum of honest signers is not permission to smuggle a bad package past the checks.

2.5 Asset identity and value sanity

Goal. The feed must be the one the consumer asked for, and the value must be usable (F4).

Built. The asset pair is checked as a **registration claim** (ADR 16): a RedStone package names a feed id and nothing else, so the binding from feed id to `(base, quote)` is established at registration and compared against the caller's expectation — matching on the base alone is still a mismatch. Value sanity is two-level (ADR 17): zero, negative or unrepresentable values reject the payload immediately, and the accepted median is converted to the canonical account's **Q64.64** fixed-point scale with the conversion, not the comparison, done last — packages are compared at one fixed width so no precision is lost before the median is taken. The result carries both scales, RedStone's and the account's, so no caller re-derives the conversion.

2.6 Typed errors, shared by both modes

Goal. Push and pull must report the same failure the same way, and a consumer must be able to act on the reason, not parse a string.

Built. One error enum covers every failure mode: malformed payload, bad signature, unauthorised signer, threshold not met, packages describing different moments, more packages than verification will pay to recover, stale, future, asset mismatch, value out of range, a price that will not fit the account's scale, an invalid configuration, and no trustworthy clock. Thirteen variants, and the suite asserts that **no two failure modes answer with the same one**. Several carry the numbers a caller would otherwise have to guess at — how many signers were met against how many required, which two timestamps disagreed. Because the enum lives in `verifier-core`, error parity between the two modes is structural rather than promised; M3 adds the test that asserts it end to end.

3. Publishing the verified price

Goal. F5: a verified price lands in the canonical RFP-019 price account — the account consumers across the LEZ estate read — populating base and quote asset, price, timestamp, source identifier and confidence interval.

Built. Two pieces. First, the account itself is **re-exported, not forked**: the M1-09 spike found the canonical struct shipped in `logos-blockchain/lez-programs` even though the grant meant to deliver it was closed, so `kanon-idl` re-exports that type and vendors its published IDL at a named commit, with a conformance test in both directions — the struct encodes to exactly the bytes a foreign decoder expects, and a seventh field would be a detected break rather than something consumers silently ignore (ADR 9, M1-26). Nothing in this repository redefines a field.

Second, `publish` — the write itself — decides what goes into each of the six fields and when an existing account may be updated (ADR 22):

field	populated with
<code>base_asset, quote_asset</code>	the registration claim, checked against the account on every update — an account for another pair is refused, never repointed
<code>price</code>	the verified median, on the account's Q64.64 scale
<code>timestamp</code>	the observation's round — the time the signers vouched for, not the time of the write
<code>source_id</code>	the constant <code>RedStone</code> , ASCII right-padded: bytes a consumer can write down and check with nothing from this repository
<code>confidence_interval</code>	zero, per the account's own documentation for a source that publishes none — <code>RedStone</code> does not

The update rules are where the security lives. Three identity fields — the pair and the source — are refused overwrites, and an account another source populated is not this adaptor's to write. A newer observation moves the price and the timestamp and nothing else; **an older observation inside the validity window cannot move the price backwards**, and replaying the same observation is not an update. So a verified-but-stale-relative-to-the-account payload — honest signatures, old round — cannot roll a fresh price back.

One decision inside ADR 22 is worth pausing on: `source_id` deliberately does *not* name the writer program. An identifier that moved with the program id would leave a rebuilt or upgraded adaptor unable to update the accounts it created — its own history would lock it out. The tests assert that rebuilding the adaptor does not change what it writes. The related upgrade question that remains genuinely open is in section 7.

What remains for M2 is the wrapper, not the write: `submit_price`, the SPEL instruction that calls `publish` inside a real transaction.

4. What an update costs

Goal. M0 priced the cryptographic primitives in a measurement harness. M1 prices the shipped code path: `verify_feed`, component by component, inside the product guest, over payloads RedStone published — so the published figures are figures about the delivered code, not a model of it (ADR 20).

Built. `COSTS.md`, generated and asserted by the same test code: the table CI compares against is printed by the code that asserts it, so a published figure and an asserted figure cannot drift apart. The method is measuring **prefixes** of the real pipeline — each stage calls exactly the functions `verify_feed` calls, in the same order — and differencing adjacent stages, so the rows add up to the total and a regression has nowhere to hide.

component	1 signer	3 signers	5 signers
decode	582	1,495	2,408
keccak256	17,476	52,428	87,380
secp256k1 recovery	585,276	1,755,580	2,922,890
signer-set membership	162	534	970
the rest of <code>verify_feed</code>	17,773	21,601	26,142
whole update	621,269	1,831,638	3,039,790

One number to reconcile before reading further. The M0 report's headline for the same scenario is **1,906,737** cycles, and this one is 1,831,638. They do not disagree; they measure different things, and the difference is worth naming because both documents travel together. M0 priced a whole LEZ transaction, including **158,290 cycles** of LEZ framework floor and instruction handling — real cost, but not the verifier's and not anything this repository can change. The table above prices `verify_feed` alone: the framework is out, and decode, the median, the staleness window and the threshold are in, none of which existed in M0's harness. Where the two overlap they agree closely — software keccak256 over 77 bytes at 17,498 cycles against 17,476 here — and the one visible gap, 19,725 cycles per recovery, is the address derivation M0's harness did not perform. `COSTS.md` carries that reconciliation in full.

At the RFP's default three signers, recovery is **95.85%** of the update, keccak256 is 2.86%, and everything M1 added around them — the median, value sanity, the staleness window, the threshold, the Q64.64 conversion — is under 1.3% combined. One more signer costs about 605,000 cycles whatever the count already is: nothing amortises, so threshold width remains linearly priced, exactly as M0 predicted.

Three consequences:

1. **M0's recommendation held on the shipped path.** The mixed accelerator configuration — recovery accelerated, keccak256 in software — is what the product guest runs, and CI asserts both choices by cycle count (ADR 6, with the measurement method in ADR 20), so neither can regress silently.
2. **The whole update fits comfortably:** 5.5% of LEZ's 33,554,432-cycle per-transaction budget at three signers. The adversarial case is the number that matters more, and it also fits: the largest payload the decoder accepts is read and verified in 23,042,390 cycles, **69% of the budget**, leaving 31% for the program doing the verifying. That is a CI-asserted ceiling, not an estimate — together with ADR 25 it means no payload, however adversarial, can cost more than one affordable transaction.
3. **The case for a recovery precompile is unchanged and now measured on product code.** Anything that halves recovery halves the update.

Still to come in later milestones: the push write's cost (M2), the pull per-read cost and byte accounting (M3), and the consolidated report (M5).

5. How we know it works

211 tests pass across the product workspace. This section is not a list of them; it is the four **kinds** of evidence and what each kind is uniquely able to show, plus the CI machinery that keeps all of it true on every push.

5.1 Conformance against payloads RedStone signed

Hand-written fixtures prove the code agrees with itself. To prove it agrees with **RedStone**, the suite verifies payloads captured from RedStone's production gateway, using their own signatures as the oracle (ADR 19): a secp256k1 signature recovers to the address RedStone published only if the signed span matches theirs byte for byte, so a decoder that misread the format by even one byte would recover nobody. A negative control drives the point home: seven plausible-but-wrong readings of the signed span all recover to nobody. The envelope framing is covered separately, with the sample payload vendored from RedStone's own Rust SDK — fifteen packages, all of which decode and recover. The capture script refuses to record a vector from a round still landing, so the fixtures cannot be internally inconsistent.

5.2 Property tests: claims about every input

Examples test the inputs someone thought of. The core's central claims are about **all** inputs — no payload panics the decoder, the threshold is exact for every M and N, the scale conversion never loses the invariants — so those are tested by generation (ADR 21), with the framework searching for counterexamples rather than checking a list. For a zkVM guest the panic-freedom property is the one that matters most: a panic is an aborted transaction, and no chosen set of examples can establish its absence.

5.3 The accept-and-reject contract, in one suite

One module reads the full accept-and-reject contract off the crate's public API (ADR 23): every acceptance the RFP requires, every rejection, and the assertion that no two failure modes collapse onto the same error variant. It is written against the public surface both modes call, which is why it is one suite rather than one per mode — `verify_feed` has no argument naming its caller, so there is no per-mode behaviour to test. It is also where the adversarial cases live: the duplicate that must not deny the feed, the stranger that must, the replayed round, the spliced payload, the package past the size ceiling.

5.4 The traceability matrix is machine-checked

`TRACEABILITY.md` maps all 36 RFP-020 requirements to tasks, implementations and evidence — and the matrix is itself a test (ADR 11). CI fails if the rendered document disagrees with its sources, if any cited test, CI job or document does not exist, or if a task id written anywhere in the repository appears in no row. A row cannot be marked `verified` without something a test run can point at. Current standing: 1 verified, 13 partial, 22 planned — the honest numbers for a

milestone that builds the core and defers the modes, and M5-06 is the task that must leave every row verified.

5.5 The gates and the sequencer

Everything above runs in CI on every push, alongside the infrastructure M1 owed:

gate	what it refuses
lint, build, test	warnings, format drift, any red test
no_std	a change that ties guest-reachable code to the standard library
guest build	product code that no longer cross-compiles to the zkVM target
licence allowlist — the cargo deny job	any dependency outside the agreed licence set, over every workspace (ADR 1)
traceability	the matrix drifting from the repository, either direction
sequencer	a standalone LEZ sequencer that no longer boots and serves
cost guardrails	any cycle-count figure moving without the change owning it (exact equality, ADR 10)

The sequencer gate deserves a sentence: CI boots a Bedrock node and a sequencer **without** the `lgs` toolchain, from a prebuilt image at the same LEZ revision the product resolves (ADR 12), which is what M2's and M3's integration tests will run against. A separate workflow proves the `lgs` build path itself stays green (ADR 28). Version pins are exact across the estate (ADR 8), with two workspaces and a lockfile per resolution domain (ADR 7) so a routine product upgrade can never silently move a figure in the delivered M0 report.

6. What M1 deliberately does not contain

The repository's full crate layout landed in M1-01 (ADR 2), so several crates exist before the milestone that fills them. Each placeholder crate says **"Not yet implemented"** in its own doc comment and names its tasks in `TRACEABILITY.md` — the empty crates are the milestone plan made visible, not work left undone. Reading the tree without this key, they are easy to mistake for gaps; this section is the key.

crate	what fills it	milestone
<code>pull-lib</code>	the public pull API over <code>verifier-core</code> , with CI asserting it carries no aggregator dependency, that the signer set comes from the consumer and never the payload, and typed-error parity with push (M3-01 to M3-04)	M3
<code>aggregator-program</code> (instructions)	the SPEL skeleton and IDL surface, <code>submit_price</code> , and the admin-gated instructions (M2-01, M2-02, M2-06 to M2-10). The <code>publish</code> write inside it is done — section 3	M2
<code>reference-consumers/aggregator-read</code>	consumer A: read the canonical account, check the pair, enforce its own <code>maxAge</code> , refuse the action on failure (M3-05)	M3
<code>reference-consumers/pull</code>	consumer B: inline pull verification against the standalone sequencer (M3-06, M3-07)	M3
<code>kanon-sdk</code>	push submit, read, pull verify, and the CLI over them (M4-11, M4-13 to M4-16; M4-12's standalone IDL artefact belongs to <code>kanon-idl</code>)	M4
<code>kanon-relayer</code>	the headless daemon: gateway fetch, heartbeat and deviation triggers, retry policy, monitoring (M4-01 to M4-08)	M4
<code>kanon-app</code>	the Basecamp mini-app dashboard (M4-18 to M4-21)	M4

`kanon-app` is the one entry that is a directory rather than a crate, and deliberately: a Basecamp mini-app is not Rust and is not a workspace member, so it holds a README stating what will land there instead of a doc comment.

6.1 The authorised-signer roster is M2, by design

One boundary is worth stating on its own, because it looks like a verification concern and is not. In M1, the authorised signer set is an **input**: the caller hands `verify_feed` the roster along with the payload, and the core enforces M-of-N against it. Where that roster is stored, who may change it, and how it is registered per feed is the admin surface, and the admin surface is M2: capturing RedStone's authorised signer sets for the chosen data service (M2-00), `register_feed` (M2-07), `update_signer_set` behind the admin authority (M2-08, M2-14), `deregister_feed` (M2-09),

and registering the five launch feeds on devnet (M2-11). M5-14 adds the operational process for tracking roster changes RedStone makes.

The split is deliberate rather than incidental. The verifier stays free of storage and authority concerns — which is what lets pull-mode consumers supply their *own* roster in M3, a property the RFP requires — and the admin gating depends on RFP-001's authority interface, which has since moved and is one of the open questions in section 7. M1-07 spiked that interface precisely so M2 can build against it, or shim it, without redesign; the move vindicates the shim rather than invalidating the spike.

7. What we need from Logos

None of these blocks M2 from starting — each has a shim, a pin or a workaround already designed — but every one of them removes a shim, a pin or a private judgement call from the estate. Five, in descending order of how much they shape the work ahead.

Two have moved since M0, in our favour, and one of those closed on its own. Every claim below about a repository outside this one was re-checked against that repository on 2026-09-16; `m0/versions.md` carries the same five in full, at the same standing.

7.1 A SPEL release, which would settle the LEZ pin with it

This is the one concrete ask, and it is narrow. The product pins `logos-co/spel` at **v0.6.0**, tagged 2026-07-15, and that tag resolves LEZ **v0.2.0**. Everything M2 wants has landed on SPEL's `main` since: PR #256 migrated SPEL to LEZ v0.2.4 (merged 2026-08-25), and PR #257 merged the extension mechanism the admin gating in M2-06 to M2-10 builds on (2026-09-11). None of it is in a release.

So: **is a SPEL release coming, and when — or is pinning `main` sanctioned?**

We ask rather than decide because a SPEL pin is not a local choice. SPEL resolves LEZ, so whatever SPEL pins is the LEZ the aggregator links, and the two have to be spelled identically or the guest links two incompatible copies (ADR 8). That coupling is also why M0's separate question — which LEZ pin is the estate standardising on — is no longer one we can usefully ask on its own. Its factual half has since been answered: testnet runs LEZ **v0.2.1**, upstream has tagged **v0.2.5-rc3**, and v0.2.5 is the next testnet target. What remains is that the repositories we depend on do not agree today:

where	LEZ	SPEL
LEZ testnet	v0.2.1	—
<code>logos-co/spel</code> v0.6.0 — our pin	v0.2.0	—
<code>logos-co/spel</code> <code>main</code>	v0.2.4	—
<code>logos-blockchain/lez-programs</code> <code>main</code>	v0.2.4	a <code>main</code> revision, untagged
this repository	v0.2.0	v0.6.0

A SPEL release is the single event that resolves all of it. Two notes for the record. `lez-programs` already pins SPEL by `main` revision rather than by tag, which is the precedent for the answer we expect. And scaffold PR #246, whose timeline M0 asked about, merged on 2026-08-10: it covers `setup`, `run` and `doctor` but not `test-node` — which no longer matters to us, because M1-04a removed the dependency on `lgs test-node` entirely (ADR 12). That question is withdrawn.

7.2 Where the admin-authority extension is going to live

RFP-001's admin authority is what M2's registration, signer-set update and deregistration instructions gate on. When M1-07 spiked it, it was `logos-co/spel` PR #212 — written, unmerged, no activity since 2026-05-20. That is still exactly true of #212, but it is no longer the whole picture: the work moved. PR #257's extension mechanism was developed on the `feat/admin_authority_m3` branch, and the admin authority now ships as a **separate consumer crate** built on that mechanism, supplying `#[admin_authority]`, `#[require_admin]` and three management instructions.

Two things follow, and both want an answer rather than a guess.

First, **is #212 superseded?** We read it as yes, and would like that confirmed, because ADR 14's shim was designed against #212's `#[require_admin(config)]` plus `AdminConfig` shape and the extension crate's shape is close but not identical — the gate now synthesises its own account parameters. Confirming it lets us drop the shim rather than carry it. Incidentally, that also settles the naming trap we flagged in M0: the RFP-001 milestone text promises `renounce_admin`, #212 implements `revoke_admin`, and the shipped crate calls it `admin_renounce`. Three names, one operation; the shipped one is what M2 will build against.

Second, and more consequential for a deliverable that has to be open-source and auditable: the three extension crates live under a **personal GitHub namespace** — `mmlado/spel-authority`, `mmlado/spel-admin-authority` and `mmlado/spel-freeze-authority` — rather than under `logos-co`. They are Apache-2.0 and actively released, so this is not a licensing problem. It is a governance one: RFP-020 delivers a program whose admin gating, the thing deciding who may change a signer set, would depend on a repository outside the Logos organisation. **Is a move to logos-co planned?** If it is, we pin the destination; if it is not, we would like that stated, so it is recorded as a deliberate estate position rather than discovered at audit.

7.3 Two licensing questions, one cheap and one positional

The cheap one, unchanged and still open: `twap_oracle_core`, `spel-framework-core` and `spel-framework-macros` ship without a `license` field in their manifests — re-checked against each repository's `main` for this report, and still missing in all three. The terms are permissive: the repositories carry MIT and Apache-2.0 licence files. But every downstream consumer currently has to make that judgement privately, and ours is recorded as three named clarifications in `deny.toml`. A one-line manifest change in each crate removes the question for the whole estate.

The positional one, whose scope is worth stating precisely because it is narrower than it first looked. M1's licence gate found LGPL-3.0 crates — `malachite` and its family — reachable through `lee_core` → `risc0-zkvm`. They arrive **only** when `risc0`'s proving feature is enabled, which is the M0 measurement workspace's configuration and not the product's: the five crates sit in `m0/Cargo.lock` and are absent from the product lockfile, so no shipped binary links them today.

That changes if a product binary ever proves locally rather than delegating — worth knowing before the relayer grows a proving path in M4. We carry it as a narrow per-crate exception rather than waving it through (ADR 1). What we want is a stated position rather than an unread exception: **is inherited LGPL accepted for the estate?**

7.4 The canonical price account is harder to depend on than it needs to be

`OraclePriceAccount` is six Borsh fields, but the crate that defines it drags `risc0` at `=3.0.5` and `uniswap_v3_math` behind it — still the case on `lez-programs` main. Splitting the account type into a standalone crate — or at minimum relaxing `=3.0.5` to `^3.0.5` — would make the canonical account usable by the external adaptors it was written for. This is closer to a defect report than a preference; what the current pins cost this repository is recorded in ADR 8 and ADR 9.

7.5 The upgrade question underneath the price account

Recorded in ADR 22 and left open on purpose: under LEZ's ownership model a program id is the hash of its binary, an account is bound for good to the id that created it, and no operation hands an account to a new binary. So **an upgraded aggregator cannot update the price accounts the old one created** — consumers would have to follow it to new addresses. Two shapes could answer this: consumers discover the account rather than pin it, or a small permanent program owns the accounts and the upgradeable one writes through it — the only arrangement that keeps the address fixed. Both belong to M2's read-path and admin work, and neither is decidable from what LEZ documents today, which defines no upgrade path at all. We would like to settle the shape together before M2 pours concrete around one of them.

8. Reproducing this

From the repository root, inside `nix-shell`:

```

cargo test --workspace --locked          # the 211 product tests, minutes
cargo test --release --locked -p kanon-methods # the cost table, built in the guest
cargo run -p traceability --bin render-traceability # regenerate TRACEABILITY.md

```

Everything section 5 describes runs on every push: `ci.yml` carries the lint, build-test, `no_std`, guest, licence, traceability and sequencer jobs; `guardrails.yml` re-asserts M0's frozen figures and the per-component cost table; `lgs-build.yml` proves the toolchain path; `lez-sequencer-image.yml` publishes the prebuilt sequencer image CI boots.

document	contents
TRACEABILITY.md	every RFP-020 requirement, its status, its tasks and its evidence — the generated, machine-checked map of what is done and what remains
COSTS.md	the full cost table, its method, and how to regenerate it
adr/README.md	all twenty-nine decision records, with a reading order and the open questions collected
m0/versions.md	every pinned version, why each is pinned that way, and the procedure for changing one

A cost-guardrail failure is a prompt to re-measure and re-publish deliberately, not necessarily a defect — but a published figure never moves without the change that owns it.